

FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of 12 JULY 2021 - Part Theory

WITH SOLUTIONS

LAST + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

SIGNATURE:

- The exam is in **written form** and consists of **two parts**:
- **Theory** (80%): Syntax and Semantics of Languages
 1. regular expressions and finite automata
 2. free grammars and pushdown automata
 3. syntax analysis and parsing methodologies
 4. language translation and semantic analysis
- **Lab** (20%): Compiler Design by Flex and Bison
- To pass the **exam**, the candidate must succeed in **both** parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass the **theory** part, the candidate must sufficiently answer all **four** parts.

PART 1 - regular expressions and finite automata

Consider the **regular expression** R below:

$$R = (a \mid bb \mid ba)^+$$

1. Find the correct **initials**, **finals** and **digrams** of R .

Ini (R) =	Fin (R) =	Dig (R) =
Choose...	Choose...	Choose...
aa, ab, ba	aa, ab, ba	aa, ab, ba
a	a	a
a, b	a, b	a, b
aa, ab, bb	aa, ab, bb	aa, ab, bb
aa, ab, ba, bb	aa, ab, ba, bb	aa, ab, ba, bb
b	b	b
aa, ba, bb	aa, ba, bb	aa, ba, bb

Fin (R) = a, b
Ini (R) = a, b
Dig (R) = aa, ab, ba, bb

ANS:

2. Is the regular expression R above **nullable** ?
- Select one:
- ☐ True
- ☐ False

ANS: NO

3. Still with reference to the above regular expression R , what is the **language** generated by its **local sets** ? Suitably **justify** your answer.

ANS: The language generated by the local sets is $\{a, b\}^+$ (universal language except the empty string), because both sets *Ini* and *Fin* coincide with the alphabet, and set *Dig* coincides with the set of all the possible digrams on the alphabet.

4. Is the language $L(R)$ generated by the above regular expression R **local** ?
- Select one:
- ☐ True
- ☐ False

ANS: NO

5. Suitably **justify** your answer to the previous question.

ANS: Language $L(R)$ does not include, for instance, strings such as b , ab and aab , and in general all the strings that end with ab .

Consider now the **numbered version** $R_{\#}$ of the above regular expression R , terminated with the end-mark symbol $\mathfrak{t}$. Notice that usually the end-mark symbol $\mathfrak{t}$ is denoted as $\neg$.

$$R_{\#} = (a_1 \mid b_2 b_3 \mid b_4 a_5)^+ \mathfrak{t}$$

Choose the **sets** necessary for the construction of the **Berry-Sethi recognizer**.

6.

Ini ($R_{\#}$)	Fol (a_1)	Fol (b_2)	Fol (b_3)	Fol (b_4)	Fol (a_5)
Choose...	Choose...	Choose...	Choose...	Choose...	Choose...
a1 b4 t	a1 b4 t	a1 b4 t	a1 b4 t	a1 b4 t	a1 b4 t
a1 b2 b4 t	a1 b2 b4 t	a1 b2 b4 t	a1 b2 b4 t	a1 b2 b4 t	a1 b2 b4 t
a1 b2 b4	a1 b2 b4	a1 b2 b4	a1 b2 b4	a1 b2 b4	a1 b2 b4
a1 b4	a1 b4	a1 b4	a1 b4	a1 b4	a1 b4
b3	b3	b3	b3	b3	b3
a5	a5	a5	a5	a5	a5
a1 b2 t	a1 b2 t	a1 b2 t	a1 b2 t	a1 b2 t	a1 b2 t

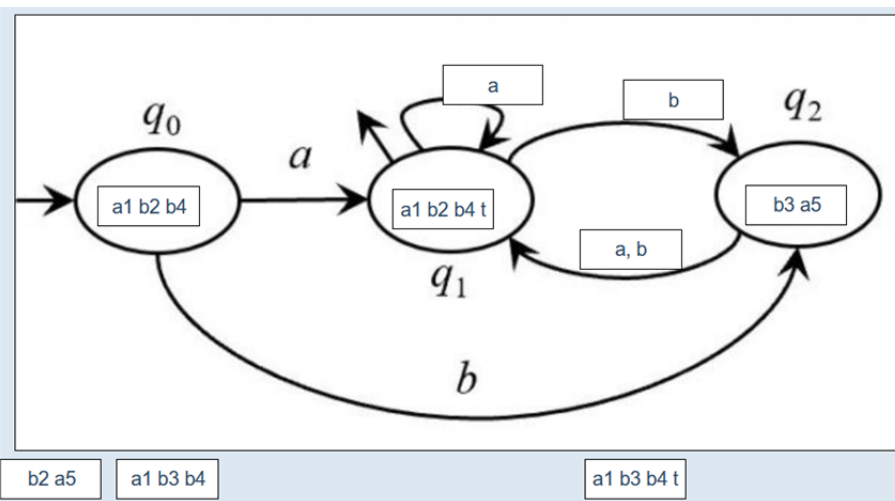
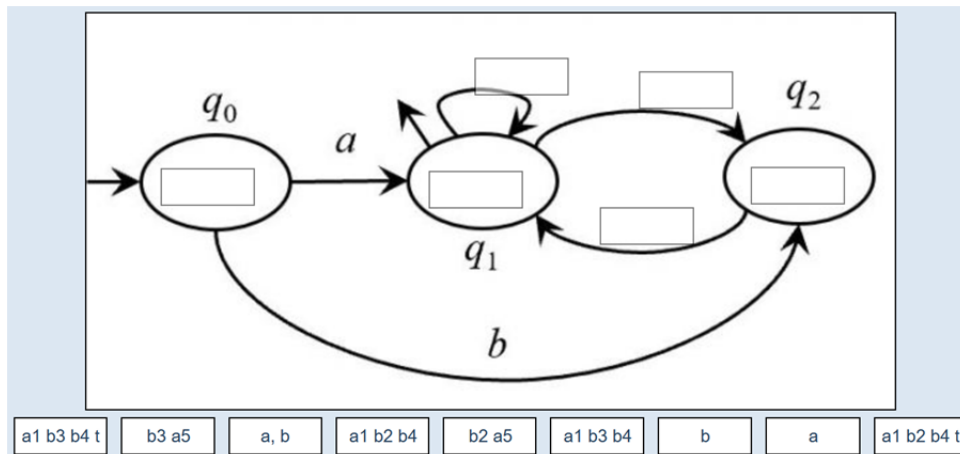
Fol (b_4)	a5	⬆
Fol (a_5)	a1 b2 b4 t	⬆
Fol (b_3)	a1 b2 b4 t	⬆
Fol (a_1)	a1 b2 b4 t	⬆
Ini ($R_{\#}$)	a1 b2 b4	⬆
Fol (b_2)	b3	⬆

ANS:

For the above regexp R , here reproduced:

$$R = (a \mid bb \mid ba)^+$$

7. **place** the correct **initials** and **followers** into the **Berry-Sethy automaton graph** below.



ANS:

8. Is the above **BS** automaton **minimal** ?

Select one:

☐ True

☐ False

ANS: **YES**

9. **Provide** a brief but exhaustive and clear **justification** to your answer to the previous question.

ANS:

state q1 is distinguishable from states q0 and q2, because the former state is final and the latter two states are not

state q0 is distinguishable from state q2, because with a transition on input b they go to the distinguishable states q2 and q1, respectively

Consider the **language** below:

$$L((a \mid bb \mid ba)^+) \cap L(((a \mid b)^2)^*)$$

10. **Write** the first **four strings** of the above language in **lexicographical** order (as usual, consider $a < b$).

ANS: *aa*, *ba*, *bb* and *aaaa*

Consider again the **language** below:

$$L((a \mid bb \mid ba)^+) \cap L(((a \mid b)^2)^*)$$

Choose the **regular expression** that defines it, among the following ones.

Select one:

- ☐ $((a \mid bb \mid ba)^2)^+$
- ☐ **none** of the other options
- ☐ $(aa \mid bb \mid ba)^*$
- ☐ $(aa \mid ab \mid ba \mid bb)^+$
- ☐ $(aa \mid bb \mid ba)^+$
- ☐ $(aa \mid bbbb \mid baba)^+$

11.

- ☐ $(aa \mid bb \mid ba)^*$
- ☒ **none** of the other options
- ☐ $(aa \mid bb \mid ba)^+$
- ☐ $((a \mid bb \mid ba)^2)^+$
- ☐ $(aa \mid ab \mid ba \mid bb)^+$
- ☐ $(aa \mid bbbb \mid baba)^+$

ANS:

12. **Provide** a suitable, brief and precise **justification** for your answer to the previous question.

ANS:

- $(aa \mid bb \mid ba)^+$ does not generate string **abba**
- $(aa \mid bbbb \mid baba)^+$ does not generate string **babb**
- $((a \mid bb \mid ba)^2)^+$ does not generate string **bb**
- $(aa \mid ab \mid ba \mid bb)^+$ generates string **baab**
- $(aa \mid bb \mid ba)^*$ generates the empty string

PART 2 - Free grammars and pushdown automata

Consider a language L , over the alphabet $\{a, "(", ")"\}$, consisting of **nested lists**, as follows:

- the depth level is arbitrary
- the list elements are denoted by letter a
- the length of a (sub)list is arbitrary and a (sub)list may be empty
- and each (sub)list is opened and closed by a pair of open and closed round brackets "(" and ")", including the top level list

Here is a possible **BNF** grammar G_1 for language L (axiom S):

$$S \rightarrow "(" X ")"$$

$$X \rightarrow \varepsilon \mid Y X$$

$$Y \rightarrow a \mid "(" X ")"$$

Sample **valid** strings: $() (a) (a(aa)) ((a)aa) \dots$

13. Is grammar G_1 **ambiguous** ?

Select one:

☐ True

☐ False

ANS: NO

14. Shortly **justify** your answer to the previous question, whether grammar G_1 is **ambiguous** or not.

ANS: Grammar G_1 is clearly deterministic **LL(1)**: the alternative rules of Y have guide sets $\{a\}$ and $\{ "(" \}$, disjoint, while those of X have guide sets $\{ ")" \}$ and $\{ a, "(" \}$, also disjoint. Therefore G_1 is not ambiguous.

Which is the **simplest type** of **automaton** that can recognize language L ?

Select one:

☐ **finite** state

☐ **nondeterministic** pushdown

☐ **deterministic** pushdown

☐ **Turing** machine

Select one:

☒ **deterministic** pushdown

☐ **nondeterministic** pushdown

☐ **Turing** machine

☐ **finite** state

15. ANS:

16. Try to **simplify** grammar G_1 , and **find a new equivalent BNF grammar** G_2 that uses only **two nonterminals**. Shortly **justify** your solution.

ANS:

Here is a possible **BNF** grammar G_2 for language L (axiom S), with two nonterminals:

$$S \rightarrow "(" X ")"$$

$$X \rightarrow \varepsilon \mid a X \mid "(" X ")" X$$

The previous sample strings are still valid. The right recursion of X generates lists, possibly empty, with elements a or sublists (X). Thus grammar G_2 is clearly equivalent to G_1 .

17. If possible, try to **simplify** grammar G_1 even more, and **find another equivalent BNF grammar G_3** that uses only **one nonterminal**. Shortly **justify** (the equivalence of) your solution, if you can find one; otherwise explain **why** a solution does not exist.

ANS: Obviously impossible: in a grammar having only one nonterminal, the top level list might unavoidably have no external brackets.

18. Consider the regular language generated by the regular expression R below, over the alphabet $\{a, "(", ")"\}$ of three symbols:
- $$R = (aa \mid "(" \mid ")")^*$$
- Basing on grammar G_1 , which generates language L , and possibly by modifying G_1 , find a **new BNF grammar G_4** that generates the **intersection** language:
- $$L \cap L(R)$$

ANS: The only constraint imposed by the intersection, is that the sequences of consecutive elements a should be of even length (including zero). Here is a possible grammar G_4 for the intersection language (axiom S):

$$S \rightarrow "(" X ")"$$

$$X \rightarrow \varepsilon \mid Y X$$

$$Y \rightarrow aa \mid "(" X ")"$$

Rule $X \rightarrow Y X$ can only generate sequences of type a^{2n} , for any n . Nothing else changes.

19. Consider the **intesection language** below, where $\overline{L(R)}$ denotes the complement language of $L(R)$:
- $$L \cap \overline{L(R)}$$
- Write** the simplest possible **EBNF** grammar for it. Shortly **justify** your answer.

ANS:

Of course, the complement of the regxep R forbids the brackets. Therefore the intersection is empty.

A simplest possible EBNF grammar is:

$$S \rightarrow \emptyset$$

otherwise, one could use a grammar with only circular derivations like, trivially:

$$S \rightarrow S$$

of course, such a grammar is not in the reduced form.

Select one:

☐ True

☐ False

20. Is the **complement language** of the language L generated by grammar G_1 , a **context-free** language ?

ANS: TRUE

21. Shortly **justify** your answer to the previous question, whether the complement language of language L is **context-free** or not.

ANS: From a previous question, we know that language L is context-free deterministic, and it is well known that the family of deterministic context-free languages is closed under complement.

22. This space is for writing your personal notes to answer other questions. These notes **will not be corrected**.

Consider a fragment of a **programming language** that exhibits the following features:

- the language is a **sequence**, possibly empty, of statements, separated by semicolon ";"
- a statement is either an **assignment** of an **arithmetic expression** to a **scalar variable**, or a **multiple-way conditional** called **select**
- an expression may contain n -ary **addition** and n -ary **multiplication** (with the usual operator precedence and unspecified associativity), numerical **constants**, scalar **variables** and **subexpressions** enclosed in round brackets
- the condition of the **select** structure is an expression, the **select** may have zero or more **branches** (cases), and each branch is tagged by a numerical constant
- optionally, there may be only one **default** branch, *not necessarily the last one*

Example (sketched):

```
alpha := beta × (gamma + delta) ;
```

```
select 2 × eta do
```

```
    branch 1 : statement list
```

```
    branch 5 : statement list
```

```
    default : statement list
```

```
    branch 2 : statement list
```

```
    ...
```

```
end select ;
```

```
...
```

Write and **EBNF** (i.e., extended) grammar **G** that generates the described programming language (axiom **PROG**), as simple as possible; the grammar may be ambiguous. For simplicity, numerical **constants** and variable **names** can be schematized by terminals **num** and **id**, respectively.

Please use the **template** below, with some nonterminals already provided (if necessary, add more nonterminals and their rules).

23.

ANS:

Grammar **G**:

<PROG> → [**<STAT>** (" ; " **<STAT>**)^{*}]

<STAT> → **<ASGN>** | **<SELECT>**

<ASGN> → id " := " **<EXPR>**

<SELECT> → **select** **<EXPR>** **do** **<BRANCH_LIST>** **end select**

<EXPR> → **<TERM>** (" + " **<TERM>**)^{*}

<TERM> → **<FACT>** (" × " **<FACT>**)^{*}

<FACT> → id | num | "(" **<EXPR>** ")"

<BRANCH_LIST> → **<BRANCH>**^{*} [**<DEFAULT>**] **<BRANCH>**^{*} // ambiguous rule

<BRANCH> → **branch** num " : " **<PROG>**

<DEFAULT> → **default** " : " **<PROG>**

The rule of nonterminal **<BRANCH_LIST>** includes the substructure **<BRANCH>**^{*} **<BRANCH>**^{*} (where the optional default is omitted), which is clearly ambiguous. If you preferred a non-ambiguous grammar, such a rule could be reformulated as follows:

<BRANCH_LIST> → **<BRANCH>**^{*} **<DEFAULT>** **<BRANCH>**^{*} | **<BRANCH>**^{*} // non-ambiguous rule

As it often happens, the ambiguous formulation is more compact and expressive, since it immediately exposes the optionality of the default, though it is less suitable for parsing.

Suppose now the keyword **end select** is **removed**. Which statement(s) below, related to such a modified programming language, is (are) **true** ?

Select one or more:

- ☐ the language has both a "dangling branch" and a "dangling default" ambiguity
- ☐ the language does not have any "dangling branch" or "dangling default" ambiguity
- ☐ the language only has a "dangling branch" ambiguity
- ☐ the language only has a "dangling default" ambiguity

24.

- ☐ the language only has a "dangling default" ambiguity
- ☐ the language does not have any "dangling branch" or "dangling default" ambiguity
- ☒ the language has both a "dangling branch" and a "dangling default" ambiguity
- ☐ the language only has a "dangling branch" ambiguity

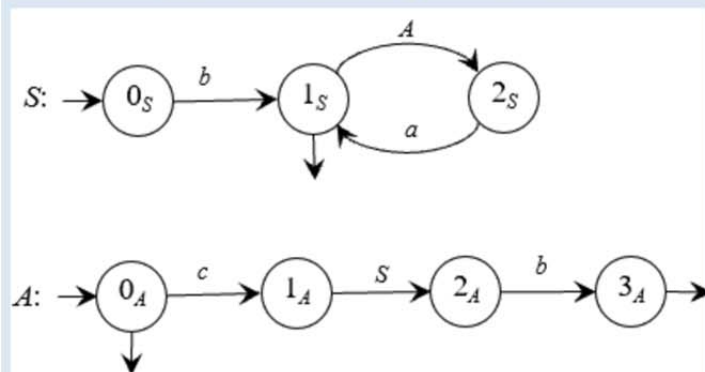
ANS:

25. Shortly **justify** the answer(s) given to the previous question, whether the modified language has any "dangling" ambiguity or not.

ANS: A select with one branch and one default is structurally identical to an *if-then-else* structure, which may have a "dangling else" ambiguity. Since the positions of default and branch may be swapped, there is a "dangling branch" ambiguity as well.

PART 3 - Syntax analysis and parsing methodologies

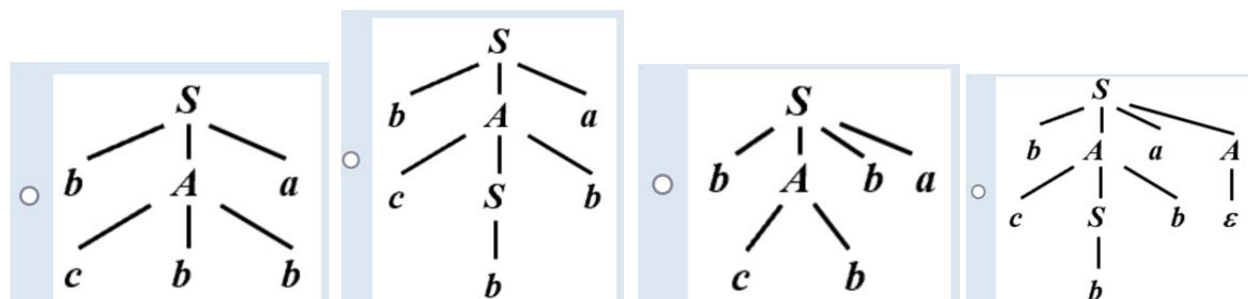
Consider the **grammar G** defined by the **machine net** below, with terminal alphabet **a, b, c**, and nonterminal alphabet **A, S** (axiom **S**):



Choose the correct **syntax tree** for the string **bcbbba**.

Select one:

26.



ANS:

☐

$$\begin{array}{c}
 S \\
 \swarrow \quad \downarrow \quad \searrow \\
 b \quad A \quad b \quad a \\
 \swarrow \quad \searrow \\
 c \quad b
 \end{array}$$

☒

$$\begin{array}{c}
 S \\
 \swarrow \quad \downarrow \quad \searrow \\
 b \quad A \quad a \\
 \swarrow \quad \downarrow \quad \searrow \\
 c \quad S \quad b \\
 \downarrow \\
 b
 \end{array}$$

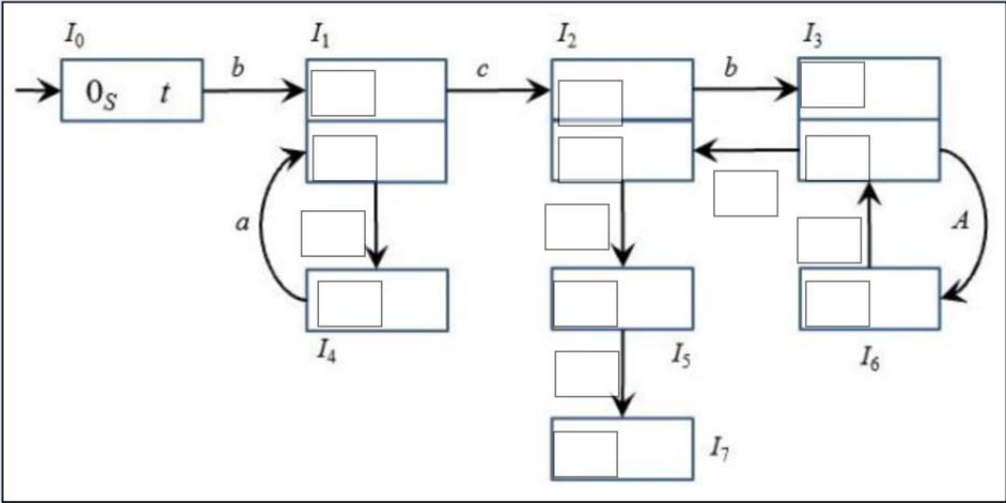
☐

$$\begin{array}{c}
 S \\
 \swarrow \quad \downarrow \quad \searrow \quad \swarrow \quad \searrow \\
 b \quad A \quad a \quad A \\
 \swarrow \quad \downarrow \quad \searrow \quad \downarrow \\
 c \quad S \quad b \quad \varepsilon \\
 \downarrow \\
 b
 \end{array}$$

☐

$$\begin{array}{c}
 S \\
 \swarrow \quad \downarrow \quad \searrow \\
 b \quad A \quad a \\
 \swarrow \quad \downarrow \quad \searrow \\
 c \quad b \quad b
 \end{array}$$

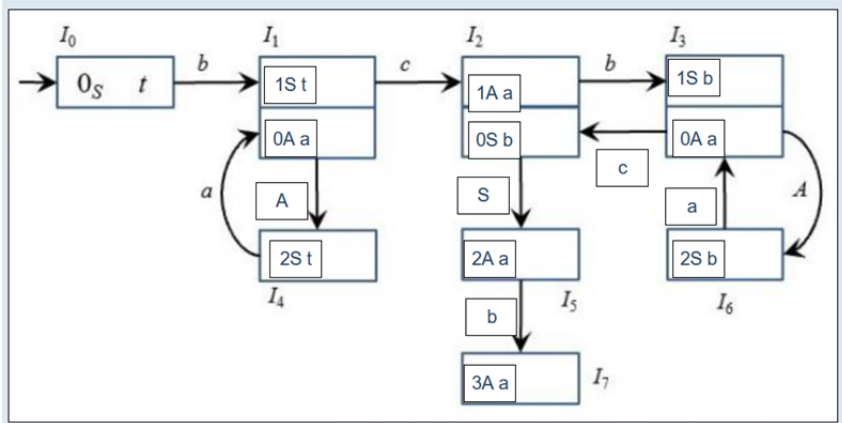
Please **fill in** the missing parts in the macro-states (m-states) of the following **pilot** automaton of the grammar **G** above (please notice that the letter **t** denotes the terminator symbol $\perp$):



0S t	A	0S a	0A t	1A a	0S b	0A a	a	0A a	1A t	2S a	3A a
2A t	S	2S t	b	c	1S b	1S a	2A a	1S t	2S b		

27.

ANS:



This space is for writing your personal notes to answer the previous questions. These notes **will not be corrected**.

28. Say if the grammar G above is of type **ELR(1)**.

Select one:

☐ True

☐ False

ANS: TRUE

For the pilot of the grammar **G** above, the valid string **bcbbba** is accepted deterministically. Simulate the execution of the **ELR syntax analyzer** of **G** and list the **stack contents** at each move (terminal shift, nonterminal shift or reduction). Use the **template** table below (first row already filled in).

↶

A ▾

B

I

☰

☷

🔗

🔗

🖼️

H-P

Fill the table below by overwriting the dots '...'. Number of rows not significant:

row #	stack contents -----	input string left to analyze	description of the analyzer move
1	0	b c b b a t	initial configuration
2	0 b 1	c b b a t	terminal shift I0 → I1 on b
3	...	...	...
4	...	...	...
5	...	...	...
6	...	...	...
7	...	...	...
8	...	...	...
9	...	...	...
10	...	...	...
11	...	...	...
12	...	...	...
13	...	...	...
14	...	...	...
15	...	...	...
16	...	...	...

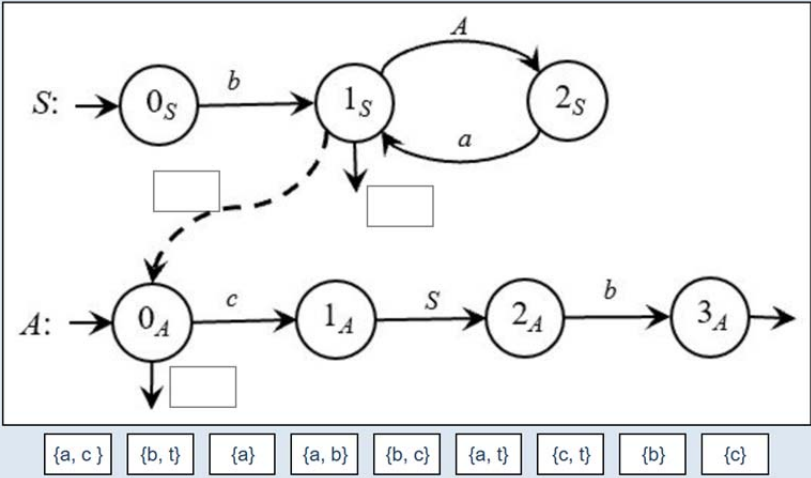
29.

ANS:

row #	stack contents -----	input string left to analyze	description of the analyzer move
1	0	b c b b a t	initial configuration
2	0 b 1	c b b a t	terminal shift I0 → I1 on b
3	0 b 1 c 2	b b a t	terminal shift I1 → I2 on c
4	0 b 1 c 2 b 3	b a t	terminal shift I2 → I3 on b
5	0 b 1 c 2 S 5	b a t	reduce b→S + n.t. shift I2→I5 on S
6	0 b 1 c 2 S 5 b 7	a t	terminal shift I5 → I7 on b
7	0 b 1 A 4	a t	reduce c S b→A + n.t. shift I1→I4 on A
8	0 b 1 A 4 a 1	t	terminal shift I4 → I1 on a
9	0 S	t	reduce b A a → S + accept
10	...	...	...

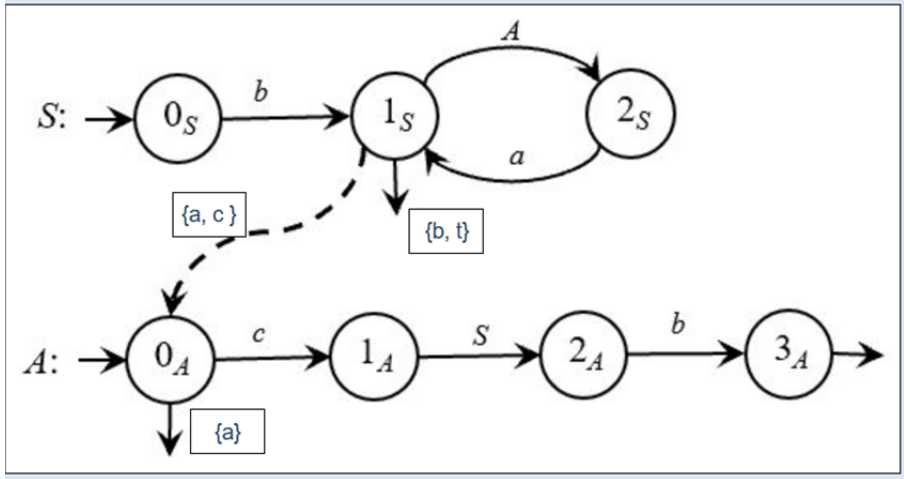
30. This space is for writing your personal notes to answer the previous questions. These notes **will not be corrected**.

Please **fill in** the guide **sets on the call arcs and exit arrows** of the machine of the grammar **G** above. Please note that **t** denotes the terminator symbol $\perp$.



31.

ANS:



32. Say if the grammar **G** above is of **type ELL(1)**. ANS: TRUE

Simulate the execution, with input the string *bcbb*_a, of the *ELL syntax analyzer* of *G* and list the *stack contents* at each move (shift, call or return).

Use the *template* table below (first row already filled in). For all the call and return moves, please indicate the relevant guide set element.

↴

A ▾

B

I

≡

≡

🔗

🔄

🖼

H-P

Fill in the simulation table below by overwriting the dots '...'. Number of rows not significant:

row #	stack contents	input string left to analyze	description of the analyzer move
1	0S	b c b b a t	initial configuration
2	...	...	...
3	...	...	...
4	...	...	...
5	...	...	...
6	...	...	...
7	...	...	...
8	...	...	...
9	...	...	...
10	...	...	...
11	...	...	...
12	...	...	...
13	...	...	...
14	...	...	...
15	...	...	...
16	...	...	...

33.

ANS:

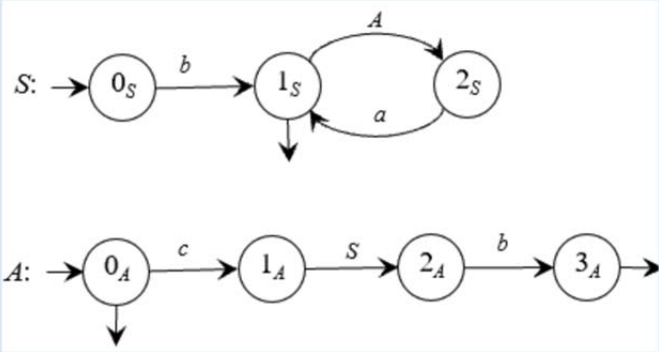
row #	stack contents	input string left to analyze	description of the analyzer move
1	0S	b c b b a t	initial configuration
2	1S	c b b a t	shift 0S → 1S on b
3	2S 0A	c b b a t	call A (guide c)
4	2S 1A	b b a t	shift 0A → 1A on c
5	2S 2A 0S	b b a t	call S (guide b)
6	2S 2A 1S	b a t	shift 0S → 1S on b
7	2S 2A	b a t	return from S (guide b)
8	2S 3A	a t	shift 2A → 3A on b
9	2S	a t	return from A (guide a)
10	1S	t	shift 2S → 1S on a
11	1S	t	acceptance condition
12			

34. This space is for writing your personal notes to answer the previous questions. These notes **will not be corrected**.

Simulate the execution, with input the string **bcbb****a**, of the **Earley syntax analyzer** of **G**, and list the **contents of the Earley vector E []** at each position, one row for each array element. The items inside the vector are to be written enclosed in parentheses, e.g., **(1S 4)**; you may highlight a final machine state in an item by formatting such a state in bold. Use the **template** below (first row already filled in).

Also **say** which is, in the Earley vector, the item that witnesses string acceptance.

Grammar **G** is reported below for your convenience.



35.

↴

A ▾

B

I

☰ ☷

🔗

🔍

🖼️

H:P

E[0] = (0S 0)

b

E[1] =

c

E[2] =

b

E[3] =

b

E[4] =

a

E[5] =

The string is accepted because

ANS:

E[0] = (0S 0)

b

E[1] = (**1S** 0) (0A 1) (2S 0)

c

E[2] = (1A 1) (0S 2)

b

E[3] = (**1S** 2) (0A 3) (2S 2) (2A 1)

b

E[4] = (**3A** 1) (2S 0)

a

E[5] = (**1S** 0) (0A 5)

The string is accepted because **state** E[0] includes the item (**1S** 0), state **1S** being an axiomatic final one.

PART 4 - Language translation and semantic analysis

The following **source grammar** G_S (axiom S):

$$S \rightarrow X S$$

$$S \rightarrow d$$

$$X \rightarrow a X$$

$$X \rightarrow b X$$

$$X \rightarrow c$$

generates a language of **two-level lists**. The top-level list is terminated by a letter d , while each bottom-level list has letters a and b as its elements, and is terminated by a letter c . The top level list may be empty, but its terminator is always present; the same applies to the bottom-level lists.

With reference to the **source language** defined by the grammar G_S above, a **translation** τ is defined as follows:

- in each bottom-level list, the elements a and b are ordered, first a then b
- each bottom-level list is initiated by a letter c , and is not terminated
- the top-level list is unchanged

Example: $\tau(abbcbaacabcbbaacd) = cabbcabcbcaabd$

Write a **target grammar** G that, combined with G_S , provides a **syntactic translation scheme** that defines translation τ .

36.

ANS:

target grammar G_t

$$S \rightarrow c X S$$

$$S \rightarrow d$$

$$X \rightarrow a X$$

$$X \rightarrow X b$$

$$X \rightarrow \epsilon$$

or scheme G_τ

$$S \rightarrow \epsilon / c X S$$

$$S \rightarrow d / d$$

$$X \rightarrow a / a X \epsilon / \epsilon$$

$$X \rightarrow b / \epsilon X \epsilon / b$$

$$X \rightarrow c / \epsilon$$

37. Is the translation τ defined before **rational**, i.e., regular, definable through a **rational translation** expression ?
- Select one:
- ☐ True
- ☐ False
- ANS: FALSE

38. Shortly **justify** your answer to the previous question, whether translation τ is **rational** or not.

Scheme G_τ clearly shows that, for ordering the elements a and b while keeping their number in each bottom-level list, a recursive self-embedding rule, i.e., $X \rightarrow b/\epsilon X \epsilon/b$, is necessary, which cannot be emulated by a rational expression.

ANS:

39. Is the translation τ defined before **deterministic**, i.e., the proposed scheme is of type **(E)LL** or **(E)LR** for some k ?
- Select one:
- ☐ True
- ☐ False
- ANS: TRUE

40. Shortly **justify** your answer to the previous question, whether translation τ is **deterministic** or not.

ANS:

The source grammar G_s is clearly of type **LL** for $k = 1$, since each alternative rule for nonterminal X is immediately initiated by a different terminal symbol, while the alternative rules for the axiom S have guide sets $\{a, b, c\}$ and $\{d\}$, respectively, which are disjoint. Furthermore, translation τ is a function (one-valued) on strings. Therefore the scheme can be computed by a recursive-descent syntactic translator and the translation is deterministic.

41. Now consider the **inverse translation** τ^{-1} . Is it **deterministic** ? Shortly **justify** your answer.

The inverse translation is definitely **not deterministic**. In fact, it is not a function either, since it is multi-valued, e.g., $\tau^{-1}(cabd) = \{abcd, bacd\}$. Therefore it cannot be deterministic.

ANS:

Now, again with reference to the **source language** defined by the grammar G_S above, a **new translation τ'** is defined as follows:

- a bottom-level list is left unchanged (elements **a** and **b**, and terminator **c**), provided it contains the same numbers of elements
- otherwise, a bottom-level list is completely removed (including its terminator **c**)
- the top-level list is unchanged (except for the removed bottom-level lists)

Example: $\tau'(abbcbaacbaacd) = bacabcd$

Write a **attribute grammar G_A** that, basing on the syntactic support G_S , defines such a new **semantic translation τ'** .

To this end, use the following **four attributes**:

- **na** and **nb** are integer **right** attributes associated with **X**, and respectively count how many elements **a** and **b** a bottom-level list contains
- **e** is a boolean **left** attribute associated with **X**, and tells if a bottom-level list contains the same numbers of elements **a** and **b**
- **t** is a string **left** attribute associated with **X** and **S**, and provides the final translation **τ'** in the root node (axiom **S**) of the syntax tree

Please **write**, in the template provided below, the **equations** (semantic functions) for the attributes associated with each rule of the attribute grammar G_A .

42.

- | | |
|------------------------------|-------|
| 1: $S_0 \rightarrow X_1 S_2$ | |
| 2: $S_0 \rightarrow d$ | |
| 3: $X_0 \rightarrow a X_1$ | |
| 4: $X_0 \rightarrow b X_1$ | |
| 5: $X_0 \rightarrow c$ | |

ANS:

1: $S_0 \rightarrow X_1 S_2$	$na_1, nb_1 := 0$	$t_0 := t_1 t_2$	
2: $S_0 \rightarrow d$	$t_0 := "d"$		
3: $X_0 \rightarrow a X_1$	$na_1 := na_0 + 1$	$e_0 := e_1$	<u>if</u> $e_0 == \text{true}$ <u>then</u> $t_0 := "a" t_1$ <u>else</u> $t_0 := \varepsilon$
4: $X_0 \rightarrow b X_1$	$nb_1 := nb_0 + 1$	$e_0 := e_1$	<u>if</u> $e_0 == \text{true}$ <u>then</u> $t_0 := "b" t_1$ <u>else</u> $t_0 := \varepsilon$
5: $X_0 \rightarrow c$		$e_0 := (na_0 == nb_0)$	<u>if</u> $e_0 == \text{true}$ <u>then</u> $t_0 := "c"$ <u>else</u> $t_0 := \varepsilon$

Select one or more:

- ☐ **none** of the two options
- ☐ of type **one-sweep**
- ☐ of type **L**

43.

Is the attribute grammar G_A that answers the previous question, of type **one-sweep** and maybe even of type **L** ?

- ☒ of type **L**
- ☐ **none** of the two options
- ☒ of type **one-sweep**

ANS:

44.

Shortly **justify** your answer(s) to the previous question, whether the attribute grammar G_A is of type **one-sweep** and maybe even of type **L**, or not.

ANS:

Grammar G_A is of type **one-sweep**: the right attributes depend only on right ones of the parent node, the boolean left attribute depends on right attributes or on itself, and the string left attribute depends only on left attributes (including itself). The one-sweep condition is easily satisfied.

Grammar G_A is even of type **L**: its syntactic support G_s is deterministic LL(1) (see a previous question) and in the rule 1, i.e., in the only rule with a right-hand side containing two nonterminals, the syntactic order of the nonterminals is the same as their semantic order, since those do not have any dependencies among sibling nodes.